

M3 · TESTING CON IA · JUE 6-AGO

De la teoría al arnés real: cerramos criterio y arrancamos el Lab

Continuación del deck del martes · desde Clean Architecture hasta la auditoría del verde.

Instituto CIRD · Diplomado IA aplicado al desarrollo

Dónde estamos · dónde vamos

El deck del martes cubrió intención → contratos → refactor. Hoy cerramos criterio + arrancamos práctica.

MAR 4	JUE 6	MAR 11	JUE 13
Fundamentos + prompts	Criterio · auditoría · Lab I	Contenido + Lab II	M4 arranca + Lab III
✓	HOY		

Formato M3: 1 hora contenido + 1 hora laboratorio guiado.

Bloques del día: Clean Architecture · Validación vs. dominio · evidencia y límites · auditoría · Lab arranca hoy.

3 capas · qué va dónde · qué NO va

El comportamiento por defecto de la IA es monolítico. Nosotros forzamos la separación.

1

Routers · app/routers/

Solo HTTP · reciben request · devuelven response · SIN lógica de negocio · SIN SQL

Test: ¿Podés cambiar de FastAPI a Flask sin tocar la lógica?

2

Services · app/services/

Lógica de negocio · cálculos · reglas · INDEPENDIENTE de FastAPI y BD

Test: ¿Podés testear la lógica sin HTTP ni BD real?

3

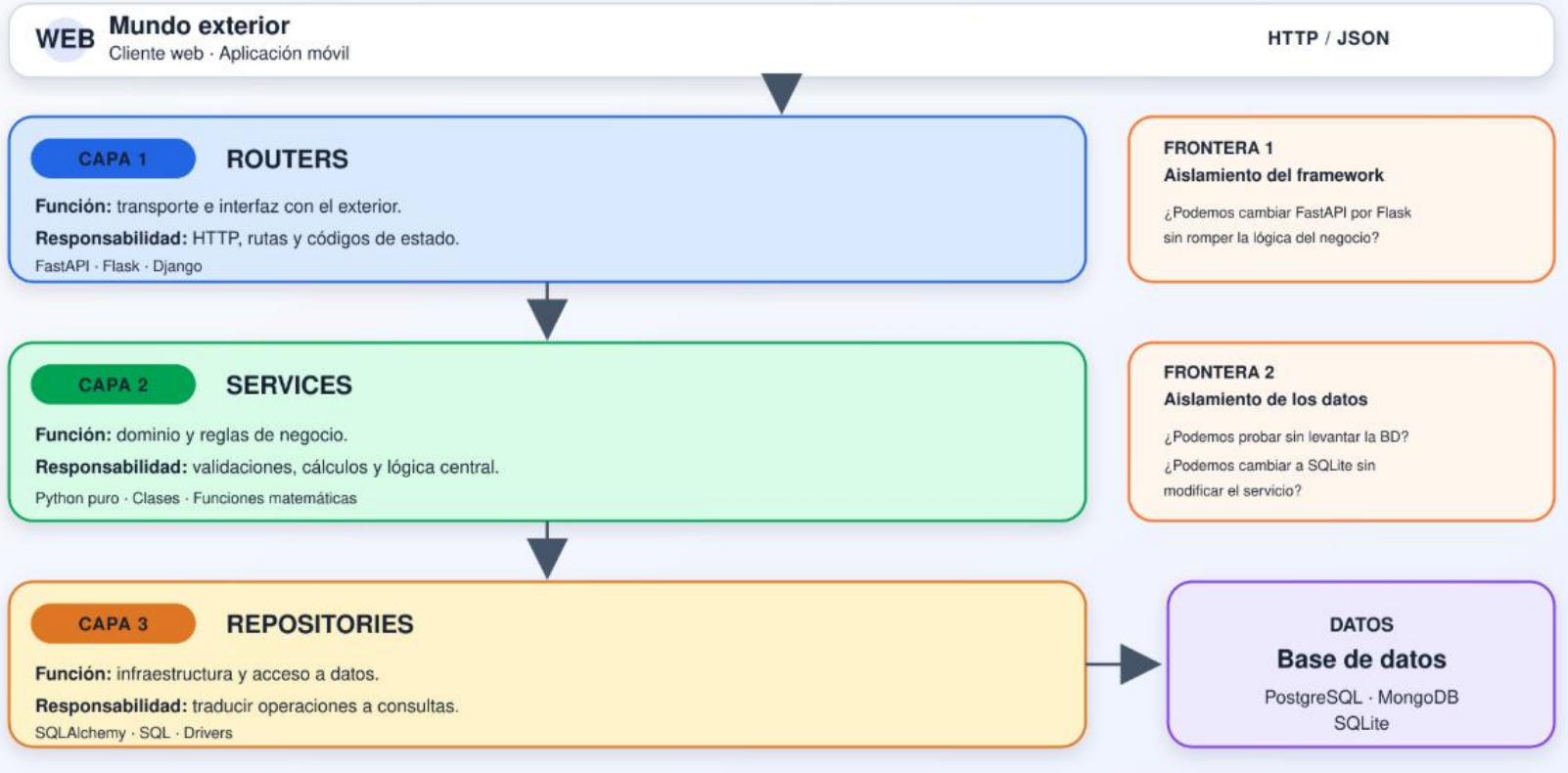
Repositories · app/repositories/

Único lugar donde se habla con SQL o APIs externas · aislado del resto

Test: ¿Podés cambiar Postgres → SQLite sin tocar services?

Anti-patrón · "espagueti asistido" · 150 líneas con validación + SQL + HTTP + reglas todo mezclado.

Arquitectura en tres capas



El orden importa · no se saltea pasos

Los LLMs son excelentes con límites claros · un test unitario es el límite más matemático.

1

Contratos (Pydantic)

PRD → modelos con validaciones estrictas

2

Tests (Pantalla Roja)

Le pedís tests para un endpoint QUE AÚN NO EXISTE · pytest debe FALLAR

3

Implementación

Le pasás los tests fallidos · le pedís código para que pasen

4

Verificación (Pantalla Verde)

pytest verde = válido · si falla · pegás el error y arregla

Pantalla ROJA es señal BUENA · significa que estás haciendo TDD auténtico (test antes que código).

Los contratos contienen la imprevisibilidad del LLM

Sin validación estricta · la IA puede procesar datos que no debería · Pydantic actúa como muro.

 `app/schemas/models.py`

```
from pydantic import BaseModel, EmailStr, Field

class TransaccionCreate(BaseModel):
    comercio_id: str = Field(min_length=36, max_length=36) # UUID
    amount: float = Field(gt=0) # positivo
    email: EmailStr
    pan_last4: str = Field(min_length=4, max_length=4, pattern=r"^\d{4}$")
```

Si el LLM intenta `guardar amount=-100 · pan_last4="abc" · email="juanperez"` · Pydantic **RECHAZA** con **ValidationError** ANTES de que llegue a la BD. La IA está contenida.

Analogía: es el arnés que sujeta al escalador · si la IA da un mal paso · el arnés lo frena.

Pydantic protege el borde · el dominio decide

Confundir ambas cosas produce tests frágiles y arquitectura acoplada.

CONTRATO DE ENTRADA

- tipo y formato
- campos requeridos
- longitud y rango local
- estructura anidada
- serialización
- rechazo temprano

Ejemplo: $\text{amount} > 0$

INVARIANTE DEL DOMINIO

- permisos y ownership
- estado actual
- relación entre entidades
- políticas y plazos
- cálculos monetarios
- consecuencias transaccionales

Ejemplo: $\text{amount} \leq \text{saldo reembolsable}$

Testear cada regla en la capa que la conoce · esquema · caso de uso · o integración.

Cientos de inputs aleatorios validando propiedades

Especialmente útil con código generado por IA · explora casos que el humano no habría pensado.

TESTING TRADICIONAL

```
def test_suma():  
    assert suma(2, 3) == 5  
    assert suma(0, 0) == 0  
    assert suma(-1, 1) == 0
```

3 casos · escritos a mano.

PROPERTY-BASED (Hypothesis)

```
@given(a=integers(), b=integers())  
def test_suma(a, b):  
    assert suma(a, b) == suma(b, a)  
    assert suma(a, 0) == a
```

100+ casos · valida PROPIEDADES (conmutatividad · elemento neutro).

En Python · biblioteca Hypothesis

TDD mejora la disciplina, con matices

La práctica es valiosa por el feedback y el diseño · no por una promesa automática.

LO QUE SÍ APORTA

- especificación mediante ejemplos
- feedback temprano
- diseño para testabilidad
- protección al refactor
- documentación ejecutable
- reducción del tamaño de cada cambio

LO QUE NO GARANTIZA

- requisitos correctos
- cobertura de todos los riesgos
- ausencia de vulnerabilidades
- mejor productividad en todo contexto
- buena arquitectura por sí sola
- tests libres de errores

La evidencia empírica muestra efectos positivos modestos en calidad y resultados variables en productividad.

Checklist para detectar defectos en tests

Los errores más peligrosos son plausibles · consistentes · y verdes.

1	Oráculo inventado	resultado esperado no validado con requisito o dominio
2	Test tautológico	repite la misma fórmula o lógica que la implementación
3	Mock excesivo	sustituye incluso el comportamiento que debía comprobar
4	Happy path dominante	omite negativos · límites · permisos y concurrencia
5	API alucinada	usa métodos · fixtures o excepciones que no existen
6	Falso verde	assertion débil · skip · xfail o test que nunca se ejecuta

Auditar significa leer la intención y provocar una falla · no mirar solamente el check verde.

El peor tipo de test

Pasa en verde, pero no evalúa en realidad

🚨 Test que pasa pero no valida nada

```
def test_create_customer():
    with patch('app.services.customer_repo.save') as mock:
        mock.return_value = {'id': 1, 'name': 'ok'}
        result = create_customer({'name': 'test'})
        assert result['id'] == 1 # ✅ pasa · pero SOLO porque el mock
                                # devuelve exactamente eso · el código
                                # real NUNCA se llamó
```

Detectar: el mock devuelve exactamente el assert · el test pasa siempre · el código real nunca se ejerce.

Corregir: usar mocks solo para lo externo (BD · APIs) · assert sobre COMPORTAMIENTO, no sobre valor mockeado.

El test alucinado da falsa seguridad · peor que un test roto (que al menos avisa).

La división de trabajo correcta protege el criterio

La IA propone y ejecuta · la persona define intención · riesgo y aceptación.

RESPONSABILIDAD HUMANA

- seleccionar comportamiento y riesgo
- aportar requisitos y ejemplos válidos
- aprobar el oráculo
- decidir qué mockear
- revisar seguridad y privacidad
- aceptar cambios de contrato

APORTE DE LA IA

- ampliar casos borde
- generar esqueleto
- proponer parametrización y fixtures
- ejecutar y clasificar fallos
- sugerir implementación mínima
- detectar duplicación y smells

Regla: la IA puede escribir el test · no debe inventar sola qué significa "correcto".

Cinco preguntas antes de aceptar el verde

La revisión debe ocurrir ANTES de integrar el test al repositorio.

1 ¿El test corresponde a un requisito · riesgo o bug claramente identificado?

2 ¿El resultado esperado proviene de una fuente independiente y válida?

3 ¿La prueba falla si altero deliberadamente el comportamiento objetivo?

4 ¿Aísla solo dependencias externas sin mockear la unidad bajo prueba?

5 ¿Incluye negativos · límites y mensajes útiles cuando algo falla?

Checklist adicional · sin datos reales · sin red inesperada · sin skips ocultos · recolectado por el motor (pytest por ej).

El Happy Path no basta

Al menos 3 casos borde relevantes al dominio · no triviales.



Strings vacíos

"" · null · whitespace · si el modelo no valida, se filtra a la BD



Números negativos o extremos

amount=-100 · age=999 · sin `gt=0` o `le=120`, pasa



Caracteres especiales / inyección

"; DROP TABLE users;--" · unicode · emojis · sin sanitización, riesgo



Límites de tamaño

String de 10K caracteres · array de 100K items · sin límites, DoS gratis



Casos de tiempo / concurrencia

Fecha en el pasado · misma request 2 veces (idempotencia)

Testear `None` no cuenta si Pydantic ya lo bloquea · sería testear el framework.

La IA propone.
Los contratos limitan.
Los tests cuestionan.
La arquitectura contiene.
El equipo responde.

Próxima etapa: convertir el PRD en pruebas observables y un cambio revisable.

LAB 3 · PARTE 1 · ~ 60 MIN

Contratos + Tests

Pantalla Roja (TDD)

sobre tu Laboratorio

Hoy arrancamos el Lab3· el martes 11 seguimos con implementación · el jueves 13 cerramos con refactor + auditoría.

Cargamos el contexto

Un buen prompt empieza antes del prompt · con los 6 elementos que vimos el martes.

1 Comportamiento	el endpoint del PRD · qué debe hacer · qué está fuera de alcance
2 Ejemplos	casos válidos · inválidos · límites (los del ADR y las Historias INVEST)
3 Interfaces	firmas · tipos · códigos HTTP esperados
4 Arquitectura	Clean Architecture · 3 capas · Pydantic v2 · pytest
5 Entorno	Python 3.12 · FastAPI · uv · estructura del repo template
6 Restricciones	sin datos reales · sin red · determinismo · no modificar producción

Abrí tu PRD.md en VS Code · lo vas a pasar como contexto (#file:PRD.md) al modelo.

Del PRD al modelo Pydantic

El mismo prompt para todos · después auditamos con las 5 preguntas de la clase pasada.

```
PROMPT-A · con Context Payload completo
#file:docs/prd/PRD.md

CONTEXTO:
Stack: Python 3.12 · FastAPI · Pydantic v2 · pytest
Clean Architecture · este archivo va en app/schemas/

TAREA:
Del PRD (Entidades e Historias), generará el modelo Pydantic v2
para el request de creación del endpoint.

REQUISITOS:
· Validaciones estrictas (EmailStr · Field(gt=0) · pattern regex)
· Comentarios que expliquen POR QUÉ cada validación
· Sin lógica de negocio (va en service · no en el schema)

RESTRICCIONES: solo código Python · sin datos reales · determinístico
```

→ [app/schemas/models.py](#) · auditá · commit · 

Tests para un endpoint QUE AÚN NO EXISTE

*El pytest **DEBE fallar** · significa que estamos haciendo TDD auténtico.*

PROMPT-B · con Context Payload completo

#file: docs/prd/PRD.md

#file: app/schemas/models.py

CONTEXTO:

Stack: Python 3.12 · FastAPI · Pydantic v2 · pytest · TestClient

El endpoint POST vivirá en app.main:app

TAREA:

Escribí 3 tests con TestClient, sin la implementación del endpoint:

1. Happy Path (201) · 2. Error 422 (Field inválido) · 3. Caso borde del PRD

REQUISITOS:

- Patrón AAA con comentarios (# Arrange · # Act · # Assert)
- Assert sobre COMPORTAMIENTO, no valor mockeado

RESTRICCIONES: sin datos reales · sin red · determinístico · solo Python

→ **tests/test_api.py** · ejecutá pytest · debe FALLAR · commit cambios .

Aplicá las 5 preguntas AHORA sobre tu test

Si algún test es tautológico · alucinado · o mockea la unidad bajo prueba · corregilo antes del checkpoint.

- 1 ¿Cada test corresponde a un requisito del PRD (Happy · Error · Borde)?
- 2 ¿El resultado esperado sale del PRD · no de lo que la IA "decidió"?
- 3 ¿Si cambio deliberadamente el modelo Pydantic · el test falla?
- 4 ¿Solo mockeás dependencias externas · o mockeaste la unidad bajo prueba?
- 5 ¿Incluye assertion sobre COMPORTAMIENTO · no sobre valor mockeado?

Corregí lo que haga falta · commit "final: tests auditados con las 5 preguntas".

Dos entradas antes de cerrar la clase

Prompt · modelo + versión · contexto · output · cambios (los 5 campos que ya vimos).

 docs/AI_USAGE.md · entradas del M3

```
## Entrada N · 6-ago-2026 · Contratos Pydantic del endpoint
**Prompt:** PROMPT-A
**Modelo:** GitHub Copilot Chat · backend Claude Fable 5 · ago-2026
**Contexto:** #file:docs/prd/PRD.md
**Output:** app/schemas/models.py con N campos + validaciones
**Cambios:** [lo que corregí de las 5 preguntas]

## Entrada N+1 · 6-ago-2026 · Tests · Pantalla Roja
**Prompt:** PROMPT-B (slide 18)
...
```

→ Rellená con TU experiencia real · commit al final.

Al terminar esta parte deberías tener: models.py + test_api.py en rojo + AI_USAGE con 2 entradas nuevas.

Siguientes pasos

El martes retomamos desde Pasos 3 y 4 (Implementación + Verificación).

- 1 Avanzá a tu ritmo con los pasos de la demo de hoy: crear el modelo Pydantic + test en rojo -> commit/push a tu rama
- 2 Verificá pytest en rojo commiteado y AI_USAGE con las 2 entradas
- 3 Módulo 2: evaluaciones pendientes abiertas hasta este domingo 23:59
- 4 Seguimos el martes, lectura adicional en Moodle

Entrega Lab 3 completa: dom 17-ago 23:59 · Discord active siempre.

C I E R R E

Modelo. Test. Rojo.

Auditamos.

El martes

pasamos al verde.

Nos vemos el martes 11-ago · más contenido + Lab Parte 2.

Discord siempre abierto.